Hoare 划分与快速排序小组报告

算法分析、C++ 实现、时间复杂度与学习心得

摘要

本报告依据《Hoare 划分的正确性》14页课堂 PPT，对《算法导论》思考题7-1进行整理。报告通过给定数组的逐轮执行过程说明双向扫描机制，利用循环不变量证明数组访问安全、返回边界有效以及左右分区之间的大小关系，并给出与该划分方式配套的 C++ 快速排序实现。在此基础上，分别分析单次划分、均衡递归、全相等输入和最坏情况下的时间与空间复杂度，最后归纳对算法正确性和实现细节的认识。

关键词：Hoare 划分；快速排序；循环不变量；递归边界；时间复杂度

1 算法分析

1.1 问题目标与记号

设待处理子数组为 A[p:r]，其中 $p < r$，区间包含两个端点。算法保存枢轴值 $x = A[p]$，返回分界下标 j，要求 $p \leqslant j < r$，且左区间 A[p:j] 中的任意元素都不大于右区间 A[j+1:r] 中的任意元素。划分结束时，两侧内部不必已经有序。

分析与题目追踪使用从1开始的下标。实现部分使用 C++ 从0开始的下标，但始终采用闭区间 [p,r]。返回的 j 表示两个子问题的边界，枢轴元素本身可能位于任一分区内。

1.2 双向扫描的基本过程

初始化 $i = p-1$、$j = r+1$。每轮先将 j 向左移动，直到 $A[j] \leqslant x$，再将 i 向右移动，直到 $A[i] \geqslant x$。若 $i < j$，就交换两处元素并继续扫描；若 $i \geqslant j$，就返回 j。两个内部循环都先移动指针再读取元素，因此初始的区间外下标不会直接用于数组访问。

停止条件包含等号。遇到与枢轴相等的元素时，扫描会停下，而下一轮仍会先移动指针。这既影响重复元素的划分效果，也关系到边界安全。

1.3 题目样例的执行过程（a）

给定数组 A = (13,19,9,5,12,8,7,4,11,2,6,21)。取 p = 1、r = 12，固定 x = 13，初始 i = 0、j = 13。每一轮外层循环的停止位置与处理如下。

轮次	i	j	扫描与处理
1	1	11	j 检查12、11；i 检查1。交换13与6。
2	2	10	j 检查10；i 检查2。交换19与2。
3	10	9	j 检查9；i 检查3至10。指针交错，返回9。

表1 每轮结束后的完整数组，数字列表示下标

状态	1	2	3	4	5	6	7	8	9	10	11	12
初始	13	19	9	5	12	8	7	4	11	2	6	21
第1轮	6	19	9	5	12	8	7	4	11	2	13	21
第2轮	6	2	9	5	12	8	7	4	11	19	13	21
第3轮	6	2	9	5	12	8	7	4	11	19	13	21

共进行3轮外层循环和2次交换。最终 i = 10、j = 9，左区间为 (6,2,9,5,12,8,7,4,11)，右区间为 (19,13,21)。左侧所有元素均不大于13，右侧所有元素均不小于13。枢轴13位于 A[11]，这说明分界下标9并不是枢轴的最终位置。

1.4 所有元素相等时的表现（b）

若6个元素都等于7，每轮两次扫描各移动一步。前3轮分别交换下标 (1,6)、(2,5)、(3,4)，第4轮得到 i = 4、j = 3，返回3。虽然数组数值没有变化，指针仍在持续向内移动。

全相等时：j = floor((p+r)/2)，两侧长度相差至多1

其中 floor 表示向下取整。教材第7.1节的 PARTITION 以末元素为枢轴，在全相等输入上返回 r，留下规模 n-1 和0的子问题。Hoare 划分则接近对半划分，能够避免这种输入造成的极端递归不平衡。该优势不意味着所有输入都能均衡划分。

1.5 正确性与终止性（c-e）

循环不变量

每次进入外层 while 循环时，A[p:i] 中每个元素均 ≤ x，A[j:r] 中每个元素均 ≥ x，且 i < j。初始化时 i = p-1、j = r+1，两端区间均为空，因此不变量成立。区间与 x 的比较均表示其中每个元素都满足相应关系。

c. 数组访问不会越界

首轮右扫描最迟在 p 停止，因为 A[p] = x；随后左扫描进入 p，立即满足 A[p] ≥ x。后续轮次中，设上一轮交换后的指针为 I < J，则 A[I] ≤ x、A[J] ≥ x。新的右扫描最迟停在 I，新的左扫描最迟停在 J。因此每次读取数组元素时，下标都位于 [p,r] 内。

d. 返回下标满足 p ≤ j < r

由扫描安全性可知 j ≥ p。若首轮就返回，则 i = p 且 i ≥ j，结合 j ≥ p 得到 j = p < r。若在后续轮次返回，j 会在首轮 j ≤ r 的基础上至少再减1，所以 j ≤ r-1。因此两侧分区均非空，长度都严格小于原区间。

e. 左侧所有元素均不大于右侧所有元素

扫描结束后，A[p:i-1] 中每个元素均 ≤ x，A[j+1:r] 中每个元素均 ≥ x，两个停点满足 A[i] ≥ x、A[j] ≤ x。若 i < j，交换后两端区间分别扩展到 i 和 j，不变量再次成立。若 i > j，则 A[p:j] 包含在已知 ≤ x 的左侧区间内；若 i = j，再由停止条件 A[j] ≤ x，也可补齐左侧结论。于是返回时有：

$$\max A[p:j] \leq x \leq \min A[j+1:r]$$

划分终止性与快速排序正确性

每轮 i 至少增加1、j 至少减少1，间距 j-i 至少缩小2；当 i ≥ j 时返回。内部扫描都有界内停止点，因此划分过程一定终止。快速排序按区间长度归纳：长度不超过1时已有序；其余情况下，两侧严格变小，递归可分别将其排好序，再结合上述分区大小关系，得到整个区间有序。交换保持元素集合及重复次数不变。

2 算法实现

以下使用 C++11 实现，输入为整型 vector，排序在原数组上完成。hoarePartition 的前提是 $p < r$。外层 quickSort 先处理空数组和单元素数组，内部递归在 $p \geq r$ 时返回，从而满足划分过程的前提。

2.1 划分函数与快速排序函数（f）

```
#include <iostream>
#include <vector>
#include <utility>
using namespace std;

int hoarePartition(vector<int>& a, int p, int r) {
    const int x = a[p];
    int i = p - 1;
    int j = r + 1;
    while (true) {
        do { --j; } while (a[j] > x);
        do { ++i; } while (a[i] < x);
        if (i >= j) return j;
        swap(a[i], a[j]);
    }
}

void hoareQuickSort(vector<int>& a, int p, int r) {
    if (p >= r) return;
    const int q = hoarePartition(a, p, r);
    hoareQuickSort(a, p, q);
    hoareQuickSort(a, q + 1, r);
}

void quickSort(vector<int>& a) {
    if (a.size() < 2) return;
    hoareQuickSort(a, 0, static_cast<int>(a.size()) - 1);
}
```

实现中的关键边界

do-while 与教材 repeat-until 对应，保证先移动再判断；循环继续条件分别是 $a[j] > x$ 与 $a[i] < x$，因而实际停止条件保留了等号。x 使用常量保存，后续交换不会改变本轮枢轴值。

左递归区间必须为 $[p,q]$，右递归区间为 $[q+1,r]$。若直接沿用另一种 PARTITION 的 $[p,q-1]$ 写法，就可能漏排 $a[q]$。本实现允许等值元素交换，不保证稳定性。

2.2 样例程序与正确性验证

将以下 main 函数接在上一页代码之后即可运行。先对副本进行一次划分，再对原数组执行完整排序，便于分别核对划分结果与最终结果。

```
int main() {
    vector<int> a = {13, 19, 9, 5, 12, 8,
                     7, 4, 11, 2, 6, 21};
    vector<int> b = a;
    const int q = hoarePartition(
        b, 0, static_cast<int>(b.size()) - 1);
    cout << "Split index (0-based): " << q << '\n';
    cout << "After partition: ";
    for (int v : b) cout << v << ' ';
    cout << '\n';

    quickSort(a);
    cout << "After quicksort: ";
    for (int v : a) cout << v << ' ';
    cout << '\n';
    return 0;
}
```

样例运行结果

```
Split index (0-based): 8
After partition: 6 2 9 5 12 8 7 4 11 19 13 21
After quicksort: 2 4 5 6 7 8 9 11 12 13 19 21
```

程序返回0起始分界下标8，对应题目中的1起始下标9。一次划分后的数组与前文追踪完全一致。编译采用 g++ 4.9.2、C++11，开启 -Wall、-Wextra、-pedantic 检查。

验证范围与结果

验证项目	检查方式	结果
小规模穷举排序	长度0至8，元素取自 {-1,0,1}；与 std::sort 比较	9,841组通过
划分性质	验证分界范围、两侧相对 x 的关系及元素保全	9,837组通过
典型与局部区间	9组典型输入，加1组指定子区间排序	全部通过

典型输入包括空数组、单元素、二元素逆序、递增、递减、全相等、重复元素、含负数以及题目样例。验证程序启用标准库调试检查。这些结果验证功能与边界处理；复杂度结论来自下一节的数学分析。

3 时间复杂度分析

3.1 单次划分：Θ(n)

设 n = r−p+1。指针 i 只向右移动，j 只向左移动，两者各自至多移动 n 次，总移动次数为 O(n)。每轮交换耗时 O(1)，轮数也不超过线性数量。返回时 i ≥ j，两次扫描合起来已经覆盖整个区间，至少需要 Ω(n) 次访问，因此单次划分时间为 Θ(n)。

3.2 快速排序的递归代价

若划分后的左区间长度为 k = j−p+1，则由 p ≤ j < r 可知 1 ≤ k ≤ n−1，两侧长度分别为 k 与 n−k。总时间满足：

$$T(n) = T(k) + T(n-k) + \Theta(n)$$

当每层划分都接近均衡时，递归树深度为 Θ(log n)，各层处理的元素总量为 Θ(n)，所以总时间为 Θ(n log n)。全相等输入属于这一情形，其递归式为：

$$T(n) = T(\mathrm{ceil}(n/2)) + T(\mathrm{floor}(n/2)) + \Theta(n)$$

ceil 表示向上取整。若划分反复产生长度1与 n−1 的两侧，例如固定取首元素且输入严格递增，则 T(n) = T(n−1) + Θ(n)，累加得到 Θ(n²)。在互异元素的输入排列等概率的标准平均分析模型下，快速排序的期望时间为 Θ(n log n)；这一结论依赖输入分布假设，不能作为任意输入的最坏情况保证。

情形	Hoare 快速排序时间	递归栈空间
持续均衡划分	Θ(n log n)	Θ(log n)
互异元素的随机排列模型	期望 Θ(n log n)	期望 Θ(log n)
全相等输入	Θ(n log n)	Θ(log n)
最坏情况	Θ(n²)	Θ(n)

3.3 与教材 PARTITION 的比较及空间开销

在全相等输入上，教材第7.1节的 PARTITION 形成 n−1 与0的递归规模，快速排序总时间为 Θ(n²)；Hoare 划分形成近似等长的两侧，总时间为 Θ(n log n)。单次 Hoare 划分只使用固定数量的变量，辅助空间为 Θ(1)，而完整递归实现还需要上表所列的调用栈空间。

4 学习心得

4.1 函数的返回值含义决定后续算法

本题最重要的认识是，划分函数的“返回一个下标”并不足以决定快速排序的写法。必须先明确这个下标表达什么，以及函数结束时保证了哪些性质。Hoare 划分返回分区边界，枢轴仍包含在某个分区中，因此递归范围应包含 [p, j] 与 [j+1, r]。把不同划分算法的返回值含义混用，即使代码形式相似，也会产生遗漏元素等逻辑错误。

4.2 循环不变量让直观过程成为可检验的证明

双指针交换的过程容易通过样例理解，但样例无法覆盖所有输入。把每轮开始时的已处理区间写成不变量后，就可以分别解释初始化为何成立、扫描跳过了什么元素、交换如何维持条件，以及退出时为什么得到目标结论。这样的分析方式也有助于检查实现：每个判断条件都应当能对应到证明中的某一步。

4.3 边界情况与等号具有实质意义

空数组、单元素和全相等数组不能只作为最后补充的测试。它们直接揭示了函数前提、递归出口以及扫描停止条件是否合理。尤其是等号，它既使扫描能够在枢轴值处停止，也使全相等输入下的两个指针稳定向内移动。下标从1起始改为0起始时，同样需要整体检查初始化和递归边界，不能只机械地修改某个常数。

4.4 性能结论应注明适用条件

“单次划分为线性时间”并不能推出“整个快速排序始终为 $\Theta(n \log n)$”。总时间还取决于递归树的形状。Hoare 划分在全相等输入上的优势是具体且可说明的，但固定首元素选枢轴仍可能出现最坏 $\Theta(n^2)$。同样，原地交换所需的常数空间也不等于整个递归算法只占常数空间，调用栈需要单独计入。

因此，本题的学习重点可以落实为一套工作方法：先明确输入输出约定，再建立正确性证明，据此编写实现，最后通过边界测试和递归分析核对结果。代码验证与理论证明相互补充，能够帮助我们更可靠地理解和使用快速排序。

参考材料

[1] 课堂演示文稿：《Hoare 划分的正确性》，14页版，第2-14页。

[2] 《算法导论》，第7章“快速排序”，思考题7-1，第123页，课程题目照片。